

TEMA 4

STATEMENTS CONCURRENTES, TIPOS DE DATOS NUMÉRICOS Y CONVERSIONES EN EL LENGUAJE VHDL



Tema 1. Análisis de los circuitos integrados digitales



Tema 2. Diseño de sistemas digitales mediante Dispositivos Lógicos Programables



Tema 3. Introducción al lenguaje VHDL



Tema 4. Statements concurrentes, tipos de datos numéricos y conversiones en el lenguaje VHDL



Tema 5. Procesos y atributos en VHDL



Tema 6. Máquinas de estados finitas (FSM) y su descripción en VHDL



Tema 7. Diseño jerárquico y parametrizable en VHDL



Tema 8. Diseño RTL en VHDL

Tema 4. Statements concurrentes, tipos de datos numéricos y conversiones en el lenguaje VHDL

❖ Objetivos

- Analizar los conceptos de transacción y evento, y comprender la diferencia entre ambos
- Comprender el concepto de concurrencia en VHDL
- Exponer las características del lenguaje concurrente de VHDL
- Comprender la sintaxis y funcionamiento de los statements concurrentes de asignación de valor a señales
- Indicar la forma de describir la lógica combinacional
- Describir los modelos temporales empleados en la simulación de los statements concurrentes
- Comprender el concepto de múltiples drivers, y la utilidad y funcionamiento de los tipos de datos resueltos
- Estudiar los tipos de datos numéricos usados en la descripción de los circuitos digitales aritméticos: sumadores, restadores, contadores, etc
- Analizar los packages numéricos que describen los tipos de datos numéricos
- Exponer los operadores definidos para cada tipo de datos
- Analizar las distintas formas de conversión de tipo de datos

Tema 4. Statements concurrentes, tipos de datos numéricos y conversiones en el lenguaje VHDL

❖ Bibliografía

- John F. Wakerly. *Diseño Digital. Principios y Prácticas*. Ed. Prentice Hall. 2001.
- William Kafir. *VHDL 101. Everything you need to know to get started*. Ed. Newnes. 2011.
- Gina R. Smith. *FPGA 101. Everything you need to know to get started*. Ed. Newnes. 2010.
- Pong P. Chu. *FPGA Prototyping by VHDL Examples*. Ed. Wiley. 2008.
- Ion Grout. *Digital Systems Design with FPGAs and CPLDs*. Ed Elsevier Ltd. 2008.
- Peter Wilson. *Design Recipes for FPGAs*. Ed. Newnes. 2007.
- David Pellerin y otros. *VHDL Made Easy!*. Ed. Mc Graw Hill. 1997.
- P. J. Ashenden. *The Student's Guide to VHDL*. Ed. Morgan Kaufmann. 2008.
- Peter J. Ashenden. *The Designer's Guide to VHDL*. Ed. Elsevier Inc. 2008.
- Douglas Perry. *VHDL*. Ed. Mc-Graw Hill. 1998.
- R. C. Cofer and Benjamin F. Harding. *Rapid System Prototyping with FPGAs*. Ed. Elsevier Inc. 2006.



1. Introducción. Simuladores VHDL

❖ Análisis de la simulación de un statement de asignación de señal

$a \leq b;$

■ Comportamiento

- ✓ El valor actual del objeto b (entrada) se asigna a la señal a (salida)

■ Proceso de simulación

- ✓ Un statement de asignación se ejecuta siempre que cambie de valor cualquier objeto de la parte derecha del statement.
- ✓ En el ejemplo se ejecuta siempre que cambie de valor b
- ✓ Al ejecutarse el statement, se temporiza una transacción (**transaction**) para la señal de salida.
 - Una transacción especifica el valor que se asigna, y el tiempo tras el que se asignará
 - En el ejemplo, la temporización es 0 ns.
 - Se puede especificar que la transacción se realice después de un tiempo concreto mediante la cláusula **after**

$a \leq b \text{ after } 5 \text{ ns};$

- ✓ Si al realizar la transacción la señal de salida cambia su valor se dice que ha ocurrido un **evento en la señal**
 - En este ejemplo siempre se produce un evento

```
Entity ejemplo is port (  
  b : in std_logic;  
  a : out std_logic  
);  
architecture Funcional  
  of ejemplo is  
Begin  
  a <= b;  
end Funcional;
```


1. Introducción. Simuladores VHDL

❖ Conclusión

- $a \leq b$;
 - ✓ Se genera una transacción y un evento con una temporización 0.
 - ✓ a toma el valor de b inmediatamente
- $a \leq b$ **after** 5 ns;
 - ✓ Se genera una transacción y un evento con una temporización de 5 ns.
 - ✓ a toma el valor de b después de 5 ns
- $a \leq b$ **and** c;
 - ✓ Si c es '1' y se produce un evento en b, se temporiza una transacción y se produce un evento en a
 - ✓ Si c es '0' y se produce un evento en b, se temporiza una transacción en a, pero no se produce un evento

1. Introducción. Simuladores VHDL

❖ Simulador de Eventos Discretos

- Los simuladores digitales VHDL utilizan la técnica de simulación de **eventos discretos**
- Esta es una técnica que permite avanzar el tiempo a intervalos variables, en función de la planificación de ocurrencia de eventos (cambio de valor de alguna señal)
- Esto significa que no se simula el comportamiento del circuito pico-segundo a pico-segundo, si no desde que ocurre un evento hasta el siguiente, donde puede pasar un pico-segundo o varios segundos
- Durante el intervalo de tiempo en el que no se produce ningún evento se mantiene el valor de todas las señales

1. Introducción. Área concurrente

❖ Área Concurrente

- En un fichero fuente VHDL el **área concurrente** corresponde a la zona comprendida entre las palabras clave **begin** y **end** de una declaración de arquitectura.

- **Ejemplo**

```
architecture arch1 of ejemplo is
```

```
    signal Reset, DivClk : std_logic;
```

```
    constant MaxCount: std_logic_vector(15 downto 0) := "10001111";
```

```
    component count port (Clk, Rst : in std_logic;  
                          Q : out std_logic_vector(15 downto 0));
```

```
    End component;
```

```
begin
```

```
    Reset <= '1' when Q = MaxCount else '0';
```

```
    CNT1: count port map(GClk, Reset, cuenta);
```

```
    Control: process(DivCLK)
```

```
    begin
```

```
        ....
```

```
    end process;
```

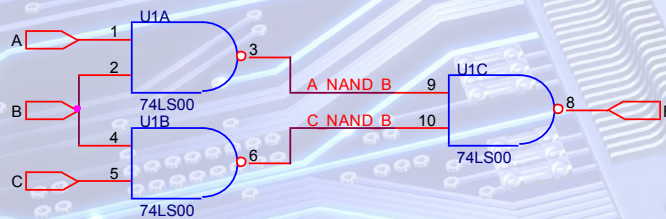
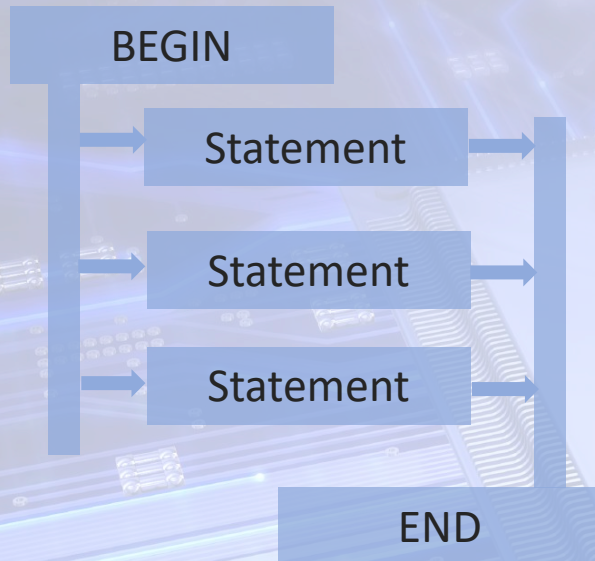
```
    ....
```

```
end arch1;
```


1. Introducción. Área concurrente

❖ Características

- Todos los statements del área concurrente **se ejecutan en paralelo** y tienen la **misma prioridad**.
- Un proceso se ejecuta también en paralelo con todos los statements y procesos declarados en el área de concurrencia.
- El orden de ejecución de los statements **no depende del de escritura en el área concurrente**



```

Architecture Funcional of puertas is
  Signal A_NAND_B, C_NAND_B : bit;
Begin
  A_NAND_B <= A nand B;
  C_NAND_B <= C nand B;
  F <= A_NAND_B nand C_NAND_B;
End;

```

```

Architecture Funcional of puertas is
  Signal A_NAND_B, C_NAND_B : bit;
Begin
  F <= A_NAND_B nand C_NAND_B;
  A_NAND_B <= A nand B;
  C_NAND_B <= C nand B;
End;

```


1. Introducción. Tiempo delta

❖ Tiempo Delta

- El tiempo delta se usa en VHDL para ordenar eventos seguidos con temporizaciones de 0 ns.
- El tiempo entre dos eventos seguidos se denomina **retardo delta**.
- El retardo delta se emplea mientras que el reloj de simulación está detenido.
 - ✓ No equivalente a un tiempo real concreto.
- En una asignación de señal con tiempo 0, la asignación de la señal no se realiza inmediatamente sino después de un retardo delta.
- $a \leq b;$
 - ✓ El valor de la señal b se asigna a la señal a después de un retardo delta.

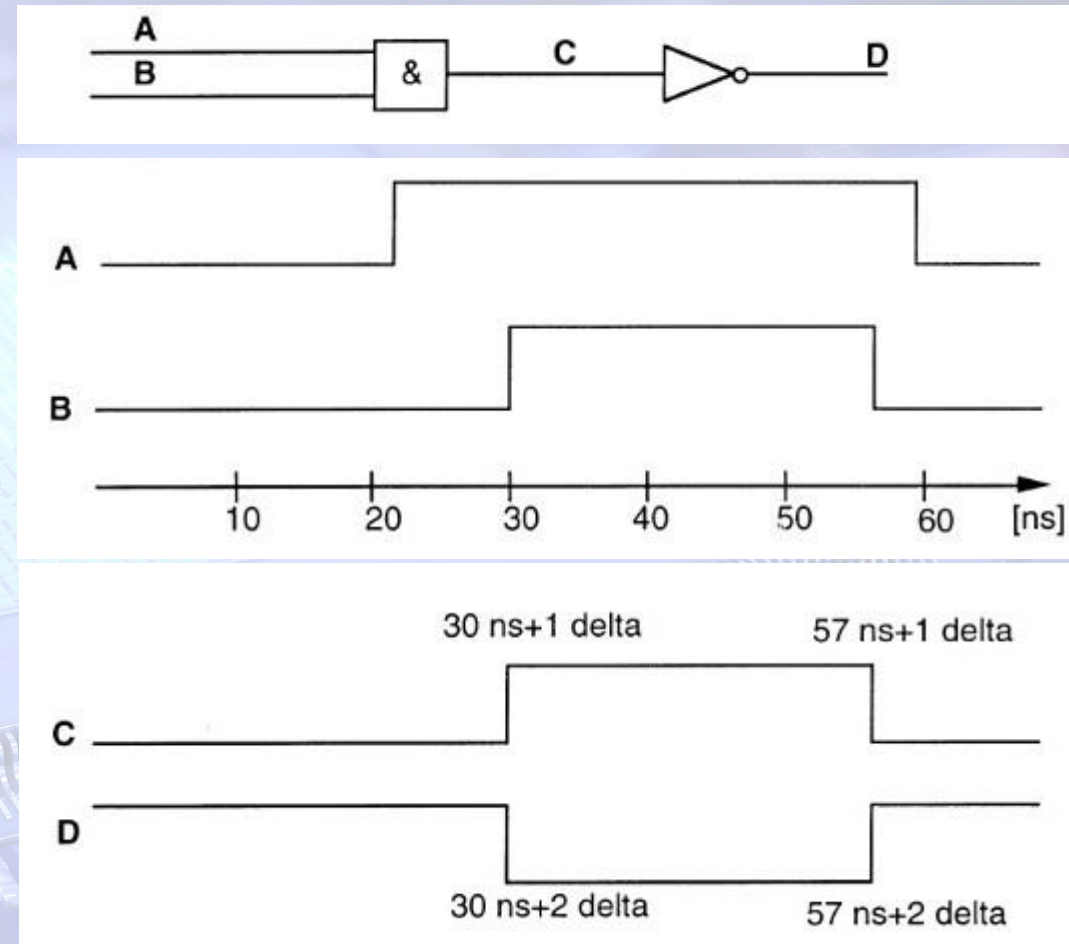
1. Introducción. Tiempo delta

❖ Ejemplo

$C \leq A \text{ and } B;$

$D \leq \text{not } C;$

- Un evento en B ($A = 1$) provoca dos eventos seguidos, primero en la señal C, y posteriormente en D
 - ✓ Las señales C y D no cambian simultáneamente.
 - ✓ El tiempo que tarda en ejecutarse una transacción es un retardo delta.



2. Statement de asignación concurrente de señal

❖ Características

- El statement de asignación es la forma más básica del estilo funcional de descripción
- Expresa el funcionamiento de un sistema asignando un valor literal, un identificador o el de una expresión a una señal

❖ Sintaxis

```
nombre_señal <= [tipo_retardo] exp_valor [after exp_tiempo];
```

- **after exp_tiempo**
 - ✓ Especifica el retardo de la asignación
 - ✓ Es opcional
 - ✓ Si no se indica VHDL considera un retardo 0
 - ✓ Solamente se emplea en la simulación

❖ Aplicaciones

- Descripción de lógica combinacional. Se emplean expresiones lógicas, aritméticas o de cualquier tipo que usen los operadores de VHDL. Es la aplicación más habitual



2. Statement de asignación concurrente de señal

❖ Ejemplo

- Puertas NAND y NOR con un retardo de propagación

```
Entity puertas is port(  
  A, B : in std_logic;  
  nan2, nor2 : out std_logic  
);
```

```
End puertas;
```

```
Architecture funcional of puertas is
```

```
Begin
```

```
  nand2 <= not (a and b) after 10 ns; -- también se puede usar
```

```
  -- el operador nand
```

```
  nor2 <= not (a or b) after 10 ns; -- también se puede usar el
```

```
  -- operador nor
```

```
End;
```


2. Statement de asignación concurrente de señal

❖ Ejemplo

- Descripción de un circuito combinacional dado por su tabla de verdad
 - ✓ Se puede describir directamente la función lógica como suma de minterms
 - ✓ La herramienta de síntesis la simplificará antes de sintetizarla:

a	b	c	F
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	0

$$F = \bar{a}\bar{b} + \bar{a}\bar{c} + \bar{b}c$$

Entity combinacional **is port**(

A, B : **in** std_logic;

F : **out** std_logic);

End combinacional;

Architecture funcional **of** combinacional **is**

Begin

F <= (not a and not b) or
(not a and not c) or
(not b and c);

End;

2. Statement de asignación concurrente de señal

❖ Especificación de retardos

- VHDL permite especificar retardos de propagación en los statements de asignación mediante la cláusula **after**
- Se distinguen dos tipos de retardos:
 - ✓ Retardo inercial (**inertial**)
 - ✓ Retardo de transporte (**transport**)
- Sintaxis

```
nombre_señal <= [tipo_retardo] exp_valor [after exp_tiempo];
```

```
Transport | [reject exp_tiempo] inertial
```

- Se puede asignar a una señal una secuencia de valores con distintos retardos

```
Y <= '1' after 10 ns, '0' after 20 ns;
```


2. Statement de asignación concurrente de señal

❖ Retardo inercial

- Es el tipo de retardo especificado por defecto en VHDL.
- Se emplea para modelar el retardo a través de una puerta.
- Indica la anchura mínima del pulso que necesita una puerta a su entrada para vencer su inercia, y por tanto, para que se produzca un evento en su salida.
- **Ejemplo. Son equivalentes**
B1 <= a **after** 10 ns;
B2 <= **inertial** a **after** 10 ns;

❖ Retardo de transporte

- Modela los retardos de los hilos.
- Propaga a la señal de salida cualquier pulso indiferentemente de su anchura.
- Este tipo de retardo se especifica mediante la palabra clave **transport**
- **Ejemplo**

B2 <= **transport** a **after** 10 ns;

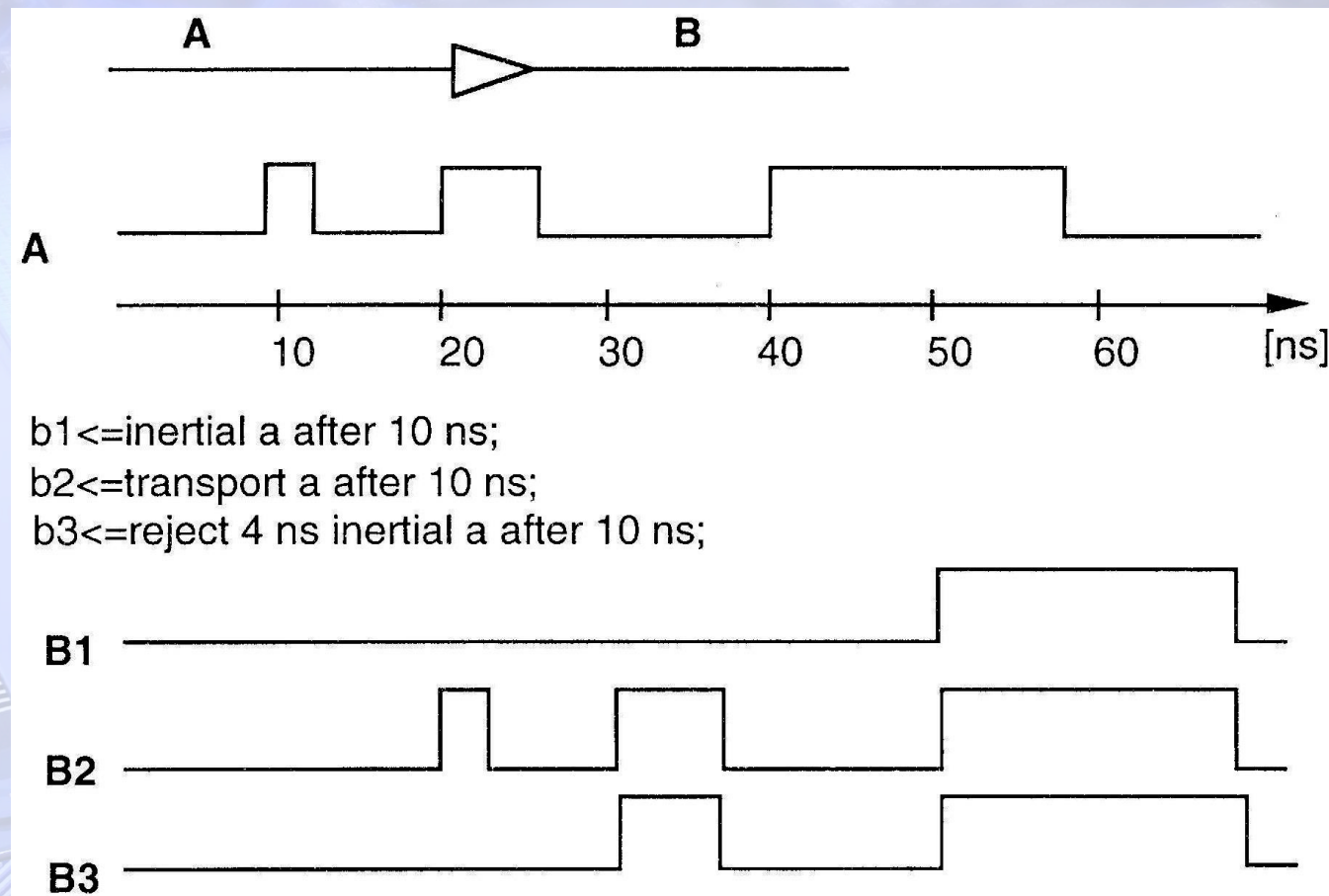
2. Statement de asignación concurrente de señal

❖ Cronograma para los distintos tipos de retardos

❖ Tiempo de rechazo (reject)

- El estándar IEEE 1076-1993 añade otra característica denominada **reject time** (tiempo de rechazo).
- Especifica la anchura mínima del pulso para que se produzca un evento en la salida para el tipo inercial.

❖ Las herramientas de síntesis ignoran las especificaciones de retardos



3. Asignación condicional de señal

❖ Descripción

- Determina qué expresión se asigna a una señal, entre varias opciones, evaluando una secuencia de condiciones

❖ Sintaxis

```
[etiqueta:] nombre_señal <= { expresión when condición else }  
                                expresión [ when condición ];
```

💡 La asignación condicional de señal permite describir condiciones con prioridad, ya que depende del orden de declaración

▪ Características

- ✓ Etiqueta es opcional y se emplea para:
 - Comentar lo que hace el statement
 - Identificar el statement durante la simulación
- ✓ **Expresión puede ser un literal, un identificador o una expresión que use los operadores de VHDL**
- ✓ Condición es una expresión lógica o la palabra clave **others**
- ✓ Se evalúan de arriba abajo las condiciones
- ✓ Las condiciones deben cubrir todos los posibles valores, si se describe un circuito combinacional
- ✓ **Se asigna un único valor a la señal**

3. Asignación condicional de señal

❖ Ejemplo

- Multiplexor de 4 a 1

```
[etiqueta:] nombre_señal <= { expresión when condición else }  
expresión [ when condición ];
```

Arquitectura equivalente

Architecture funcional of mux4a1 is

Begin

```
Y <=  E0 when Sel = "00" else  
      E1 when Sel = "01" else  
      E2 when Sel = "10" else  
      E3;
```

End;

Entity mux4a1 is port(
 Sel : **in** std_logic_vector(1 **downto** 0);
 E0, E1, E2, E3 : **in** std_logic;
 Y : **out** std_logic);
End mux4a1;

Architecture funcional of mux4a1 is

Begin

```
Y <=  E0 when Sel = "00" else  
      E1 when Sel = "01" else  
      E2 when Sel = "10" else  
      E3 when others;
```

End;

3. Asignación condicional de señal

❖ Características

▪ VHDL-87

- ✓ Debe terminarse el statement con la cláusula **else**
- ✓ Las cláusulas **when others** al final son opcionales solamente en el estándar 1076-93
- ✓ No todas las herramientas de síntesis o de simulación son compatibles con el estándar 1076-93

❖ Ejemplo

▪ Unidad lógica

architecture concurrente **of** ejemplo **is**
begin

F <= a **and** b **when** sel = "00" **else**
a **or** b **when** sel = "11" **else** --Expresión else. Válida en VHDL-87
not c;

F1 <= a **and** b **when** sel = "00" **else**
a **or** b **when** sel = "11" **else**
not c **when others**; --Expresión sin else. Solo válida en VHDL-93

End concurrente;

3. Asignación condicional de señal

❖ Características

- Se puede especificar retardos
- Un retardo distinto para cada asignación

❖ Ejemplo

▪ Unidad lógica

- ✓ Se puede especificar un retardo distinto para cada operación

architecture concurrente of ejemplo is
begin

F <= a and b after 10 ns when sel = "00" else
a or b after 15 ns when sel = "11" else
not c after 8 ns;

End concurrente;

4. Asignación seleccionada de señal

❖ Descripción

- Es similar a la asignación condicional, excepto que en la asignación seleccionada **las condiciones de entrada no implican una prioridad**

❖ Sintaxis

```
with expresión_selección select  
  identificador <= [tipo_retardo]  
    { expresión when elección, }  
    expresión when elección ;
```

- Dependiendo del valor de **expresión_selección** se asigna a la señal (identificador) una expresión. Puede ser:

```
identificador | valor | expresión con operadores de VHDL
```

- Elección es un valor de expresión_selección. Puede ser:

```
identificador | valor | rango_discreto | others
```

- Solamente se asigna una única expresión

- 💡 Se deben especificar todos los valores de expresión_selección
- 💡 Se puede emplear la cláusula **others** para especificar más fácilmente todos los valores
- 💡 Realiza la asignación por defecto

4. Asignación seleccionada de señal

❖ Ejemplo

▪ Multiplexor de 4 a 1

with expresión_selección **select**
 identificador <= [tipo_retardo]
 { expresión **when** elección, }
 expresión **when** elección ;

```
Entity mux4a1 is port(  
  Sel : in std_logic_vector(1 downto 0);  
  E0, E1, E2, E3 : in std_logic;  
  Y : out std_logic );  
End mux4a1;  
Architecture funcional of mux4a1 is  
Begin  
  with Sel Select  
    Y <=  E0 when "00",  
          E1 when "01",  
          E2 when "10",  
          E3 when others;  
End;
```


4. Asignación seleccionada de señal

❖ Características

- Se puede especificar retardos para cada expresión
- Se puede asignar una expresión para **varios valores** de la expresión de selección
 - ✓ Se usa el operador | para hacer la suma lógica de los valores

```
Entity mux3a1 is port(  
    Sel : in std_logic_vector(1 downto 0);  
    E0, E1, E2 : in std_logic;  
    Y : out std_logic );  
End mux4a1;  
Architecture funcional of mux3a1 is  
Begin  
    with Sel Select  
        Y <=  E0 when "00" | "01",  
              E1 when "10",  
              E2 when others;  
End;
```


4. Asignación seleccionada de señal

❖ Ejemplo

- La asignación seleccionada facilita la descripción de las tablas de verdad

a	b	c	F
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	0

```
Entity tabla_verdad is port(  
    a, b, c : in std_logic;  
    F : out std_logic );  
End tabla_verdad;  
  
Architecture funcional of tabla_verdad is  
    Signal sel: std_logic_vector(2 downto 0);  
    Begin  
        sel<=a & b & c;  
        with sel Select  
            F <= '1' when "000" | "001" | "010" | "101",  
                '0' when others;  
    End;
```


5. Asignación condicional o seleccionada

❖ ¿Cuándo se emplea un tipo u otro de asignación?

- La asignación condicional establece una **prioridad** en las condiciones
- **Ejemplo**

```
Y <= "01" when A = '1' else  
      "10" when B = '1' else  
      "11" when C = '1' else  
      "00";
```



La entrada A tiene
mayor prioridad

```
With std_logic_vector'(A, B, C) select
```

```
Y <= "01" when "100",  
      "01" when "101",  
      "01" when "110",  
      "01" when "111",  
      "10" when "010",  
      "10" when "011",  
      "11" when "001",  
      "00" when others ;
```


5. Asignación condicional o seleccionada

❖ ¿Cuándo se emplea un tipo u otro de asignación?

- Depende de si la descripción necesita especificar una prioridad en las condiciones
- Si más de una entrada puede estar activa
 - ✓ Asignación condicional
- Solamente se activa una entrada en un instante dado
 - ✓ Asignación seleccionada

```
Y <= "01" when A = '1' else  
      "10" when B = '1' else  
      "11" when C = '1' else  
      "00";
```

```
With std_logic_vector'(A, B, C) select  
  Y <= "01" when "100",  
        "01" when "101",  
        "01" when "110",  
        "01" when "111",  
        "10" when "010",  
        "10" when "011",  
        "11" when "001",  
        "00" when others ;
```


5. Asignación condicional o seleccionada

❖ Codificador binario

- Codificador con prioridad $\Rightarrow$ asignación condicional

```
library ieee;  
use ieee.std_logic_1164.all;  
entity codif_prioridad is port  
  ( a : in std_logic_vector(0 to 3);  
    p : out std_logic;  
    y : out std_logic_vector(1 downto 0));  
end codif_prioridad;  
architecture codif_prioridad of codif_prioridad is  
begin  
  y <= "00" when a(0) = '1' else  
        "01" when a(1) = '1' else  
        "10" when a(2) = '1' else  
        "11" when a(3) = '1' else  
        "00";  
  p <= '1' when a = X"0" else '0';  
End;
```


5. Asignación condicional o seleccionada

❖ Codificador binario

- Codificador sin prioridad $\Rightarrow$ asignación seleccionada

```
library ieee;  
use ieee.std_logic_1164.all;  
entity codif_sin_prioridad is port  
  ( a : in std_logic_vector(0 to 3);  
    p : out std_logic;  
    y : out std_logic_vector(1 downto 0));  
end codif_sin_prioridad;  
architecture Funcional of codif_sin_prioridad is  
begin  
  with a select  
    y <= "00" when "1000",  
        "01" when "0100",  
        "10" when "0010",  
        "11" when "0001",  
        "00" when others;  
    p <= '1' when a = X"0" else '0';  
End;
```


6. Driver de señal y tipos de datos resueltos

❖ Driver de señal

- Es un **circuito conceptual** que se crea al realizar una **asignación a una señal**
- Por tanto, si se asignan **múltiples valores** a una señal en el área de concurrencia, se crea una situación de **múltiples drivers** para esa señal
- **Ejemplo. Descripción incorrecta**



En otros lenguajes como ABEL-HDL sería correcto

```
-- Unidad lógica
-- Sel = 0 realiza la operación nor
-- Sel = 1 realiza la operación nand
architecture funcional of Unidad_logica is
begin
```

```
y <= (a nand b) and Sel;
y <= (a nor b) and not Sel;
```

```
End;
```


6. Driver de señal y tipos de datos resueltos

Descripción errónea

```
-- Unidad lógica
-- Sel = 0 realiza la operación nor
-- Sel = 1 realiza la operación and
architecture funcional of Unidad_logica is
begin
    y <= (a nand b) and Sel;
    y <= (a nor b) and not Sel;
End;
```

Descripción correcta

```
architecture funcional of Unidad_logica is
    Signal nand2, nor2 : std_logic;
begin
    nand2 <= (a nand b) and Sel;
    nor2 <= (a nor b) and not Sel;
    Y <= nand2 or nor2;
End;
```

❖ Driver de señal

- Se pretende describir una Unidad Lógica cuya salida Y funcione como una puerta nand o nor, según el valor de Sel
- Cada asignación a la señal Y crea un driver
 - ✓ Por tanto, se crea una situación de **múltiples drivers** para la señal Y
- **Solución**
 - ✓ Describir la asignación de valores a la señal Y mediante un único statement de asignación

Descripción correcta

```
architecture funcional of Unidad_logica is
begin
    Y <= a nand b when Sel = '1'
        else a nor b;
End;
```


6. Driver de señal y tipos de datos resueltos

❖ Tipos de datos resueltos

- En los sistemas electrónicos digitales hay casos en los que se crea una situación de múltiples drivers, como por ejemplo un **bus de datos**
- Se conecta varios drivers de tres estados al bus
 - ✓ En cualquier instante de tiempo dado sólo se puede habilitar un driver
 - ✓ El driver habilitado forzará el valor del bus y el resto pondrán un valor de alta impedancia ('Z') en su salida
 - ✓ En VHDL, aunque todos los drivers excepto uno tengan sus salidas en 'Z', se crea una situación de múltiples drivers
- **Solución**
 - ✓ Usar un tipo de dato resuelto, como el **std_logic** o el **std_logic_vector**, para describir la señal del bus

Descripción de un bus

architecture funcional of bus_datos is
begin

Bus_datos <= A when Ena1 = '1'
 else 'Z';

Bus_datos <= B when Ena2 = '1'
 else 'Z';

End;

6. Driver de señal y tipos de datos resueltos

❖ Tipos de datos resueltos: std_logic

- Si se asigna varios valores al mismo tiempo a una señal de tipo resuelta, se llama a una **función de resolución**
 - ✓ Esta analiza los valores asignados y **resuelve un único valor**, que es el que se le asigna finalmente
- Los tipos de datos definidos en el estándar **1076-87 no son resueltos**, por lo que no se pueden usar para describir un bus
- Los tipos **std_logic** y **std_logic_vector** son resueltos
 - ✓ El std_logic se define como un tipo resuelto a partir del std_ulogic.
 - ✓ Si una señal del tipo **std_logic** tiene **múltiples drivers**, la **resolución del valor** se realiza según la tabla

Ejemplos de resolución

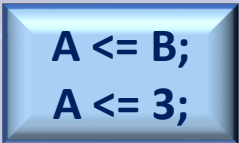
- Si se asigna **'0'** y **'1'** el valor resuelto es **'X'**
- Si se asigna **'0'** y **'Z'** el valor resuelto es **'0'**
- Si se asigna **'1'** y **'Z'** el valor resuelto es **'1'**

	'U'	'X'	'0'	'1'	'Z'	'W'	'L'	'H'	'-'
'U'	'U'	'U'	'U'	'U'	'U'	'U'	'U'	'U'	'U'
'X'	'U'	'X'	'X'	'X'	'X'	'X'	'X'	'X'	'X'
'0'	'U'	'X'	'0'	'X'	'0'	'0'	'0'	'0'	'X'
'1'	'U'	'X'	'X'	'1'	'1'	'1'	'1'	'1'	'X'
'Z'	'U'	'X'	'0'	'1'	'Z'	'W'	'L'	'H'	'X'
'W'	'U'	'X'	'0'	'1'	'W'	'W'	'W'	'W'	'X'
'L'	'U'	'X'	'0'	'1'	'L'	'W'	'L'	'W'	'X'
'H'	'U'	'X'	'0'	'1'	'H'	'W'	'W'	'H'	'X'
'-'	'U'	'X'	'X'	'X'	'X'	'X'	'X'	'X'	'X'

7. Conversión de tipo de datos

❖ VHDL es un lenguaje fuertemente tipado

- No se puede asignar directamente un valor literal o un objeto de un tipo de datos a un objeto de otro tipo distinto
- Debe realizarse una **conversión de tipo**



```
A <= B;  
A <= 3;
```

❖ Opciones de VHDL para conversión de tipo

- Datos relacionados
 - ✓ VHDL realiza la conversión automáticamente
 - ✓ Tipos de conversión
 - **Conversión explícita**
 - **Conversión implícita**
- Datos no relacionados
 - ✓ VHDL permite usar o crear una **función de conversión de tipo**

7. Conversión de tipo de datos. Conversión automática

❖ Conversión explícita de datos

- Se realiza entre tipos de datos estrechamente relacionados
- **Concepto de tipos de datos estrechamente relacionados**
 - ✓ Dos tipos de datos están estrechamente relacionados si son:
 - Tipos numéricos abstractos (integers o reals)
 - Tipos de array que tienen la misma dimensión y comparten el mismo tipo de elemento base. La dirección es indiferente
- Se realiza especificando el tipo de dato al que se convierte

❖ Conversión implícita

- Se realiza entre dos subtipos que comparten el mismo tipo base
- No se indica nada
 - ✓ La conversión se realiza automáticamente

7. Conversión de tipo de datos. Conversión automática

❖ Ejemplos de conversión explícita e implícita de datos

```
architecture ejemplo of conv_tipo is
  type array1 is array(0 to 7) of bit;
  type array2 is array(7 downto 0) of bit;
  subtype array3 is bit_vector(0 to 7);
  subtype array4 is bit_vector(7 downto 0);
  signal a1 : array1;
  signal a2 : array2;
  signal a3 : array3;
  signal a4 : array4;
  signal num_entero : integer;
  signal num_real : real;
```

```
begin
  -- a2 <= a1;
  a2 <= array2(a1);           --Conv. explícita de tipo
  -- a4 <= a3;
  a4 <= a3;                   --Conv. implícita
  -- num_entero <= num_real;
  num_entero <= integer(num_real); --Conv. explíc. de tipo
  -- num_real <= num_entero;
  num_real <= real(num_entero);   --Conv. explíc. de tipo
End;
```


7. Conversión de tipo de datos. Funciones de conversión

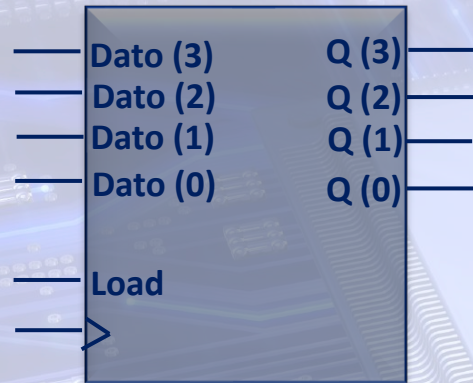
❖ Funciones de conversión de tipo de datos

A <= 3;

- Convierten datos de un tipo a otro que no están relacionados, como por ejemplo, un integer a un std_logic_vector.
- Alternativas para obtener las funciones de conversión de tipo:
 - ✓ Librería del estándar IEEE
 - ✓ Librería específica del vendedor
 - ✓ Funciones de usuario de conversión de tipo
- Una función de conversión de tipo utiliza como entrada el valor de un objeto de un tipo de datos concreto y devuelve el valor equivalente en otro tipo de datos.

7. Conversión de tipo de datos. Funciones de conversión

❖ Ejemplo. Descripción de un contador con carga paralela



```
entity contador16 is
  port (Clk, Load: in std_logic;
        Dato: in std_logic_vector(3 downto 0);
        Q : out std_logic_vector(3 downto 0);
  end contador16;
```

- Para describir el funcionamiento respecto a la cuenta se debe usar el operador de suma (+)
 - ✓ No está permitido para el `bit_vector` ni el `std_logic_vector`
 - ✓ Si está permitido para el tipo de dato `integer`
- **Conclusión:**
 - ✓ Se declaran los puertos del tipo de dato `std_logic_vector`
 - ✓ Se declara una señal (Cuenta) del tipo `integer` para definir la condición de cuenta mediante el operador +
 - Para describir la **operación de carga** se debe **convertir** las entradas de datos (Dato) a **integer** mediante una función de conversión de tipo
 - Se asigna la señal de cuenta (Cuenta) al puerto de salida de la cuenta (Q)
 - Se debe **convertir de integer a std_logic_vector**.

7. Conversión de tipo de datos. Funciones de conversión

❖ Función que convierte un std_logic_vector a un integer sin signo

```
function to_uint (vector: std_logic_vector) return integer is
    variable av: std_logic_vector (vector'length downto 1);
    variable val: integer:= 0;
    variable b: integer:= 1;
Begin
    av := vector;
    for i in 1 to vector'length loop
        if (av(i) = '1') then           --si el bit es '1',
            val := val + b;             --suma el peso correspondiente al bit
        end if;
        b:= b * 2;                      --Calcula el peso del siguiente bit
    end loop;
    return val;
end to_uint;
```


7. Conversión de tipo de datos. Funciones de conversión

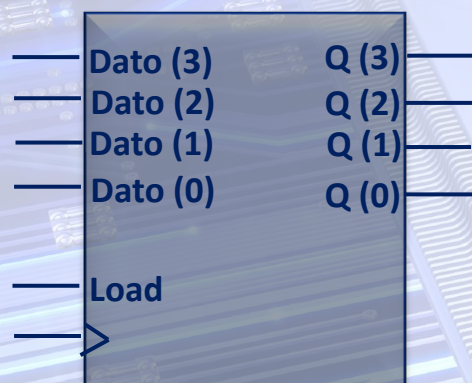
❖ Función que convierte un integer a un std_logic_vector

```
function to_vector (size: integer; val: integer) return std_logic_vector is
    variable vec: std_logic_vector (1 to size);
    variable a: integer;
begin
    a := val;
    for i in size downto 1 loop
        if a rem 2 = 1 then
            vec(i) := '1';
        else
            vec(i) := '0';
        end if;
        a := a / 2;
    end loop;
    return vec;
end to_vector;
```

-- Determina los dígitos en base 2
-- mediante el operador resto
-- también se puede usar el op. mod

7. Conversión de tipo de datos. Funciones de conversión

Descripción del contador usando ambas funciones



```

use work.conversion.all;
entity contador16 is
    port (Clk, Load: in std_logic;
          Dato: in std_logic_vector(3 downto 0);
          Q : out std_logic_vector(3 downto 0);
    end contador16;
architecture funcional of contador16 is
    signal Cuenta: integer range 0 to 15;
begin
    process(Clk)
    begin
        if Clk = '1' and Clk'event then
            if Load = '1' then
                Cuenta <= to_uint(Dato);
            elsif Cuenta = 15 then
                Cuenta <= 0;
            else
                Cuenta <= Cuenta + 1;
            end if;
        end if;
    end process;
    Q <= to_vector(4,Cuenta);
end;

```

--Carga paralela
--Convierte vector a integer
-- fin de cuenta
-- vuelve a la cuenta inicial
-- Incrementa la cuenta
--Convierte integer a vector

8. Tipos de Datos Numéricos. Packages

- ❖ El **estándar 1076-87** no define las operaciones aritméticas para los arrays de bit (`bit_vector`).
- ❖ Es recomendable usar los tipos de datos `std_logic` y `std_logic_vector` para describir diseños portables.
- ❖ El **estándar 1164**, que especifica el tipo `std_logic_vector`, tampoco define las operaciones aritméticas para este tipo de datos.
- ❖ VHDL permite definir mediante `packages` nuevas operaciones para el tipo `std_logic` y `std_logic_vector`.
- ❖ Se han desarrollado los siguientes `packages`:
 - **Numeric_std**
 - **Std_logic_unsigned**
 - **Std_logic_signed**
 - **Std_logic_arith**

8. Tipos de Datos Numéricos. Packages

❖ Recomendaciones:

- Por razones históricas los packages **Std_logic_signed**, **Std_logic_unsigned** y **Std_logic_arith** se encuentran frecuentemente en la librería IEEE
- Por el contrario, estos packages no son estándar
 - ✓ Solamente está estandarizado el package **numeric_std**
 - Se describe en el estándar **IEEE 1076.3-1997**
- Para realizar diseños portables se debe usar el package **numeric_std**

❖ Normas de empleo:

- Para usar cualquier package, se debe incluir antes de cada unidad de diseño (entidad o package) los statements:
 - ✓ **Library** ieee;
 - ✓ Un statement **use** con el package correspondiente
 - **Use** ieee.numeric_std.all;

8. Estándar Numérico 1076.3

- ❖ El **estándar 1076.3** se desarrolló con los siguientes objetivos:
 - **Declarar tipos de datos estándar y sus operaciones** para implementar circuitos aritméticos y representar datos numéricos
 - **Definir funciones de conversión de tipo** entre estos datos y los de los estándares 1076-87 y 1076-93
- ❖ El estándar 1076.3 del IEEE define el **package numeric_std**, que declara:
 - Los tipos de datos numéricos **signed** y **unsigned**, basados en el tipo de dato **std_logic (resolved)**
 - Sus correspondientes operaciones y funciones aritméticas.
- ❖ El **package numeric_std** permite el uso de operaciones aritméticas con los tipos de datos lógicos estándar (**std_ulogic** y **std_logic**) de una forma indirecta a través de los tipos **signed** y **unsigned**
- ❖ Incluye funciones adicionales, tales como **std_match**, que simplifican el empleo de los valores indiferentes

9. Tipos de Datos Numéricos. Operadores

❖ Operadores:

- Todos los packages numéricos definen nuevos operadores que se clasifican en las siguientes categorías:
 - ✓ Operaciones aritméticas
 - ✓ Operaciones relacionales
 - ✓ Operaciones de desplazamiento
 - ✓ Funciones de conversión de tipo
 - ✓ Otros (Extensiones de arrays y valores absolutos)

9. Tipos de Datos Numéricos. Operadores

❖ Operadores aritméticos

Operación	Tipos de operandos	Tipos de resultado	Package
+	Signed, unsigned, integer, natural	Signed, unsigned	Numeric_std
-	Signed, unsigned, integer, natural	Signed, unsigned	Numeric_std
*	Signed, unsigned, integer, natural	Signed, unsigned	Numeric_std
/	Signed, unsigned, integer, natural	Signed, unsigned	Numeric_std
rem	Signed, unsigned, integer, natural	Signed, unsigned	Numeric_std
mod	Signed, unsigned, integer, natural	Signed, unsigned	Numeric_std

- ⚠ No se puede operar **unsigned** con **signed**
- ⚠ Por el contrario, se puede operar **unsigned** con **natural** o **entero**
 - El tipo de dato del resultado es **unsigned**
- ⚠ Lo anterior es aplicable al tipo **signed**

9. Tipos de Datos Numéricos. Operadores

❖ Operadores aritméticos

Operación	Tipos de operandos	Tipos de resultado	Package
+	Signed, unsigned, integer, std_logic	Signed, unsigned, std_logic_vector	Std_logic_arith
-	Signed, unsigned, integer, std_logic	Signed, unsigned, std_logic_vector	Std_logic_arith
*	Signed, unsigned	Signed, unsigned, std_logic_vector	Std_logic_arith
+	Std_logic, integer, std_logic_vector	std_logic_vector	Std_logic_unsigned
-	Std_logic, integer, std_logic_vector	std_logic_vector	Std_logic_unsigned
*	std_logic_vector	std_logic_vector	Std_logic_unsigned

 No se ha incluido en la tabla el package **Std_logic_signed**

 Se permiten los mismos operadores y tipos de datos que en el package **Std_logic_unsigned**, pero con signo

9. Tipos de Datos Numéricos. Operadores

❖ Operadores relacionales

Operación	Tipos de operandos	Tipos de resultado	Package
>	Signed, unsigned, integer, natural	Boolean	Numeric_std
<	Signed, unsigned, integer, natural	Boolean	Numeric_std
=	Signed, unsigned, integer, natural	Boolean	Numeric_std
>=	Signed, unsigned, integer, natural	Boolean	Numeric_std
<=	Signed, unsigned, integer, natural	Boolean	Numeric_std
/=	Signed, unsigned, integer, natural	Boolean	Numeric_std

9. Tipos de Datos Numéricos. Operadores

❖ Operadores relacionales

Operación	Tipos de operandos	Tipos de resultado	Package
>	Signed, unsigned, integer	Boolean	Std_logic_arith
<	Signed, unsigned, integer	Boolean	Std_logic_arith
=	Signed, unsigned, integer	Boolean	Std_logic_arith
>=	Signed, unsigned, integer	Boolean	Std_logic_arith
<=	Signed, unsigned, integer	Boolean	Std_logic_arith
/=	Signed, unsigned, integer	Boolean	Std_logic_arith
>	std_logic_vector, integer	Boolean	std_logic_unsigned
<	std_logic_vector, integer	Boolean	std_logic_unsigned
=	std_logic_vector, integer	Boolean	std_logic_unsigned
>=	std_logic_vector, integer	Boolean	std_logic_unsigned
<=	std_logic_vector, integer	Boolean	std_logic_unsigned
/=	std_logic_vector, integer	Boolean	std_logic_unsigned

9. Tipos de Datos Numéricos. Operadores

❖ Operadores de desplazamiento

Operación	Tipos de operandos	Tipos de resultado	Package
Shift_Left	1º Signed o unsigned. 2º natural	Signed, unsigned	Numeric_std
Shift_Right	1º Signed o unsigned. 2º natural	Signed, unsigned	Numeric_std
Rotate_Left	1º Signed o unsigned. 2º natural	Signed, unsigned	Numeric_std
Rotate_Right	1º Signed o unsigned. 2º natural	Signed, unsigned	Numeric_std
SHL	1º Signed o unsigned. 2º integer	Signed, unsigned	Std_logic_arith
SHR	1º Signed o unsigned. 2º integer	Signed, unsigned	Std_logic_arith
SHL	Ambos Std_logic_vector	Std_logic_vector	Std_logic_unsigned
SHR	Ambos Std_logic_vector	Std_logic_vector	Std_logic_unsigned



Los operadores de desplazamiento usan dos operandos:

a <= Shift_left(a, 2); a <= SHL(a, "10");

- El operando de la derecha es del tipo natural, entero o std_logic_vector según el package usado
- Este operando indica el número de desplazamientos

9. Tipos de Datos Numéricos. Operadores

❖ Funciones de conversión

Función	Tipos de operandos	Tipos de resultado	Package
To_bitvector	Std_logic_vector	Bit_vector	Std_logic_1164
To_stdLogicVector	Bit_vector	Std_logic_vector	Std_logic_1164
To_integer	Signed, unsigned	Integer, natural	Numeric_std
To_unsigned	natural, std_logic_vector	unsigned	Numeric_std
To_signed	integer, std_logic_vector	signed	Numeric_std
Conv_integer	Signed, unsigned, std_ulogic	integer	Std_logic_arith
Conv_signed	Signed, unsigned, std_ulogic, integer	signed	Std_logic_arith
Conv_unsigned	Signed, unsigned, std_ulogic, integer	unsigned	Std_logic_arith
Conv_std_logic_vector	Signed, unsigned, std_ulogic, integer	Std_logic_vector	Std_logic_arith
Conv_integer	Std_logic_vector	integer	Std_logic_unsigned



Las funciones de conversión de entero y natural a signed, unsigned y std_logic_vector usan dos operandos de entrada:

To_unsigned(Num_entero, Num_bits)

 El operando de la derecha indica el número de bits del unsigned

9. Tipos de Datos Numéricos. Operadores

❖ Otras funciones disponibles

Función	Tipos de operandos	Tipos de resultado	Package
Rising_edge	Signal std_logic	Boolean	Std_logic_1164
Falling_edge	Signal std_logic	Boolean	Std_logic_1164
Std_match	Signed, unsigned, std_logic_vector, std_ulogic_vector	Boolean	Numeric_std
Resize	Signed, unsigned, natural	Signed, unsigned	Numeric_std
EXT	std_logic_vector	std_logic_vector	Std_logic_arith
SXT	std_logic_vector	std_logic_vector	Std_logic_arith
ABS	Signed	signed	Std_logic_arith

9. Tipos de Datos Numéricos. Operadores

❖ Funciones Rising_edge y Falling_edge

- Se declaran el package **std_logic_1164**
- Permiten detectar flancos en una señal:
 - ✓ **Rising_edge** $\Rightarrow$ flanco de subida
 - ✓ **Falling_edge** $\Rightarrow$ flanco de bajada
- **Formato de utilización**
 - ✓ **Rising_edge(identificador_señal)**
 - ✓ Devuelve un valor **boolean**
 - ☐ **True** si se ha producido un flanco de subida en la señal
 - ☐ **False** si no se ha producido un flanco de subida en la señal

▪ Ejemplo. Flip-flop activo por flanco de subida

```
Process(Clk)
```

```
Begin
```

```
  If Rising_edge(Clk) then
```

```
    Q <= D;
```

```
  End if;
```

```
End process;
```

-- Devuelve un valor true si Clk = $\uparrow$

-- Se asigna D a Q si Clk = $\uparrow$

-- de lo contrario $Q^{v+1} = Q^v$

9. Tipos de Datos Numéricos. Operadores

❖ Función Std_match

- Se declara en el package **numeric_std** y, en algunas herramientas, en el package **std_logic_arith**
- Se usa para tratar el valor '-' del tipo std_logic como indiferente y poderse usar en las operaciones de comparación, cuando haya bits indiferentes.

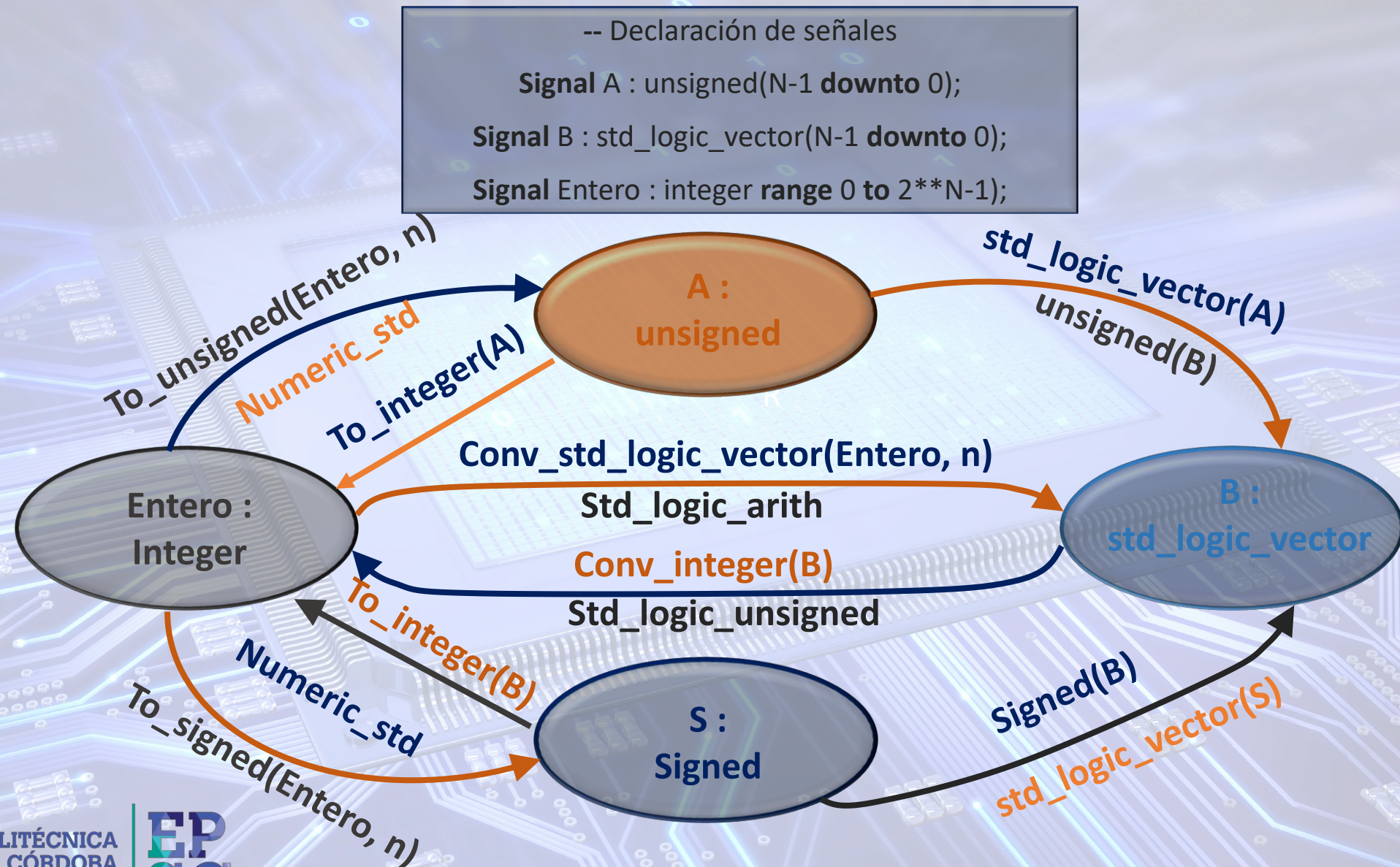
▪ Formato de utilización

- ✓ Std_match(identificador1, identificador2)
- ✓ Identificador puede ser una señal o un literal (valor)

▪ Ejemplo. Comparación con uno o varios bits indiferentes

- ✓ If a = "1 - - 1" then
 - Siempre devuelve el valor **false** salvo que **a** sea "1 - - 1"
 - Ya que las herramientas de simulación y síntesis lo interpretan como el carácter '-'
 - Aunque '-' representa el valor indiferente no se puede usar para indicar que un bit o varios no se deben tener en cuenta en la comparación
- ✓ If std_match(a, "1 - - 1") then
 - **Std_match** devuelve un valor true si se compara '-' con '1' o '0'
 - '-' indica que un bit no se debe tener en cuenta en la comparación

10. Ejemplos de conversión de tipos de datos



10. Ejemplos de conversión de tipos de datos

-- Declaración de señales

Signal A : unsigned(N-1 **downto** 0);

Signal B : std_logic_vector(N-1 **downto** 0);

Signal Entero : integer **range** 0 **to** 2**N-1);

-- Ejemplo de conversión de tipos de datos

-- A <= B;

A <= unsigned(B); -- Conv. explícita

-- B <= A;

B <= std_logic_vector(A); -- Conv. explícita

-- Entero <= A;

Entero <= To_integer(A); -- Numeric_std

-- Entero <= B;

-- std_logic_unsigned

Entero <= Conv_integer(B);

-- Numeric_std

Entero <= To_integer(unsigned(B));

-- Ejemplo de conversión de tipos de datos

-- A <= Entero;

A <= To_unsigned(Entero,N); -- Numeric_std

-- B <= Entero;

-- std_logic_arith

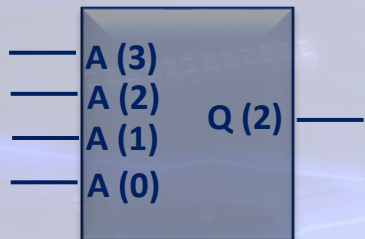
B <= Conv_std_logic_vector(Entero, N);

-- numeric_std

B <= std_logic_vector(To_unsigned(Entero, N));

10. Ejemplos de conversión de tipos de datos

❖ Generación automática de los valores de entrada en un test bench



- Supongamos un circuito combinacional cuyo comportamiento se describe mediante una **tabla de verdad de 4 variables de entrada**, que se representa mediante el array **A**
- Mediante un **bucle for** se genera todos los valores de entrada para la **simulación de un circuito combinacional**

• Process del test bench

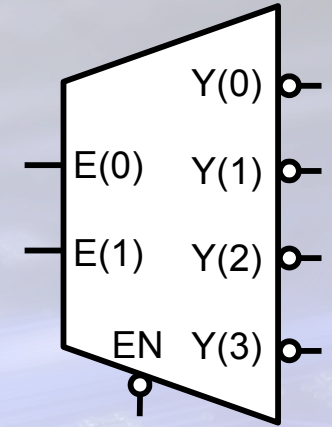
```
process
begin
  for i in 0 to 15 loop
    -- A <=
    std_logic_vector(to_unsigned(i,4));
    A <= Conv_std_logic_vector(i,4);
    wait for 100 ns;
  end loop;
end process;
```

- Los valores de entrada varían del 0 al 15
- Estos se generan mediante la variable **i** del bucle **for** que es de tipo entero
- Se debe convertir **i** a **std_logic_vector** de 4 bits antes de asignársela a las entradas de la tabla de verdad, **A**
 - Si se usa la función de conversión **to_unsigned** se debe incluir el package **numeric_std**
 - Si se usa la función de conversión **Conv_std_logic_vector** se debe incluir el package **std_logic_arith**

11. Problemas

1. Describese en VHDL un **decodificador binario de 2 a 4** con entrada de habilitación, **EN**. La entrada de habilitación y las salidas son activas a nivel lógico bajo. Las salidas del decodificador se declararán como un **std_logic_vector**. De esta forma se simplifica la descripción del decodificador
2. Describese en VHDL un **comparador de magnitud de n bits** con entrada de habilitación **EN**. Tendrá tres salidas **A_mayor_B**, **A_igual_B** y **A_menor_B** para indicar el resultado de la comparación. La entrada de habilitación y las salidas son activas a nivel lógico alto
3. A la señal **Y**, que es del tipo de datos **std_logic**, se le asignan al mismo tiempo los valores: '1', 'L', 'H' y 'Z'. ¿Qué valor se le asignará finalmente a la señal?

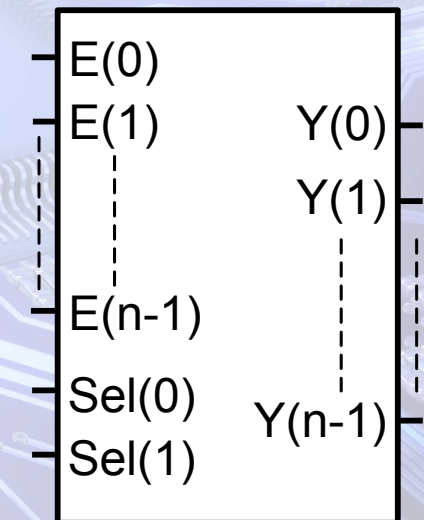
Indíquese el proceso de resolución



11. Problemas

4. Describese en VHDL un comparador de magnitud de 3 números (A, B y C) de n bits. Tendrá dos salidas, Mayor y Menor. Mayor se pondrá a 1, si $A > B > C$, y Menor se pondrá a 1, si $A < B < C$.
5. Describese en VHDL el desplazador de barril de la figura. La salida Y es igual a la entrada E desplazada a la izquierda tantos bits como indique la entrada Sel. Siempre se realiza la operación de desplazamiento lógico a la izquierda. Téngase en cuenta que el desplazador de barril es un circuito combinacional y que no se puede usar los operadores de desplazamiento analizados en el tema 3, ya que no están definidos para el tipo de datos `std_logic_vector` en el package `std_logic_1164`.

Desplazador de barril de n bits



11. Problemas

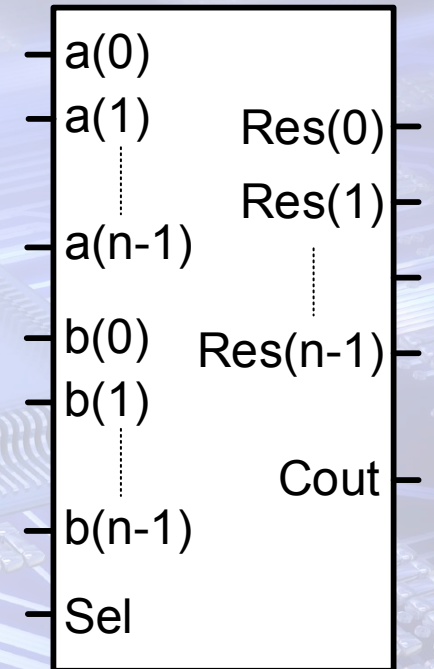
6. Sumador binario de n bits

- Describir en VHDL el sumador binario de n bits de la figura. **A** y **B** son las entradas de n bits en las que conecta los operandos a sumar. **Sum** es el resultado de la suma de n bits de A y B, y **Cout** la salida de acarreo. Los puertos A, B y Sum se declararán del tipo `std_logic_vector`, y Cout del tipo `std_logic`
- **Dos opciones:**
 - ✓ **Diseño no estandarizado**, aunque válido para la mayoría de herramientas
 - Se usa el package `std_logic_unsigned`
 - Se declaran los port del tipo `std_logic_vector`
 - ✓ **Diseño estandarizado**
 - Se usa el package `numeric_std`
 - Se declaran los port del tipo `std_logic_vector`
 - Se convierte a `unsigned` los operandos de entrada al sumarlos
 - Se convierte a `std_logic_vector` el resultado al asignarlos al port de salida
 - Como los tipos `std_logic_vector` y `unsigned` están estrechamente relacionados se usa la **conversión explícita de datos**

11. Problemas

7. Sumador-restador binario de n bits

- **A** y **B** son las entradas de n bits en las que conecta los operandos de entrada.
- **Sel** es la entrada de control que determina la operación a realizar, de forma que si está a 0 se selecciona la suma de A y B.
- **Res** es la salida de n bits en la que se representa el resultado de la operación indicada por la entrada Sel.
- **Cout** es la salida de acarreo (suma) o de préstamo (resta).
- Realícese la descripción teniendo en cuenta que el sumador-restador debe poder operar tanto con números binarios sin signo como con signo representados en el sistema de representación en C2.
- Se determinará si falta alguna señal de entrada y/o de salida en la representación a nivel de bloque de la figura, y de ser así, incluirlas en el diseño.



11. Problemas

❖ Sumador-restador binario de n bits

■ Modos de funcionamiento:

Sel = 0 $\Rightarrow$ Res = A + B

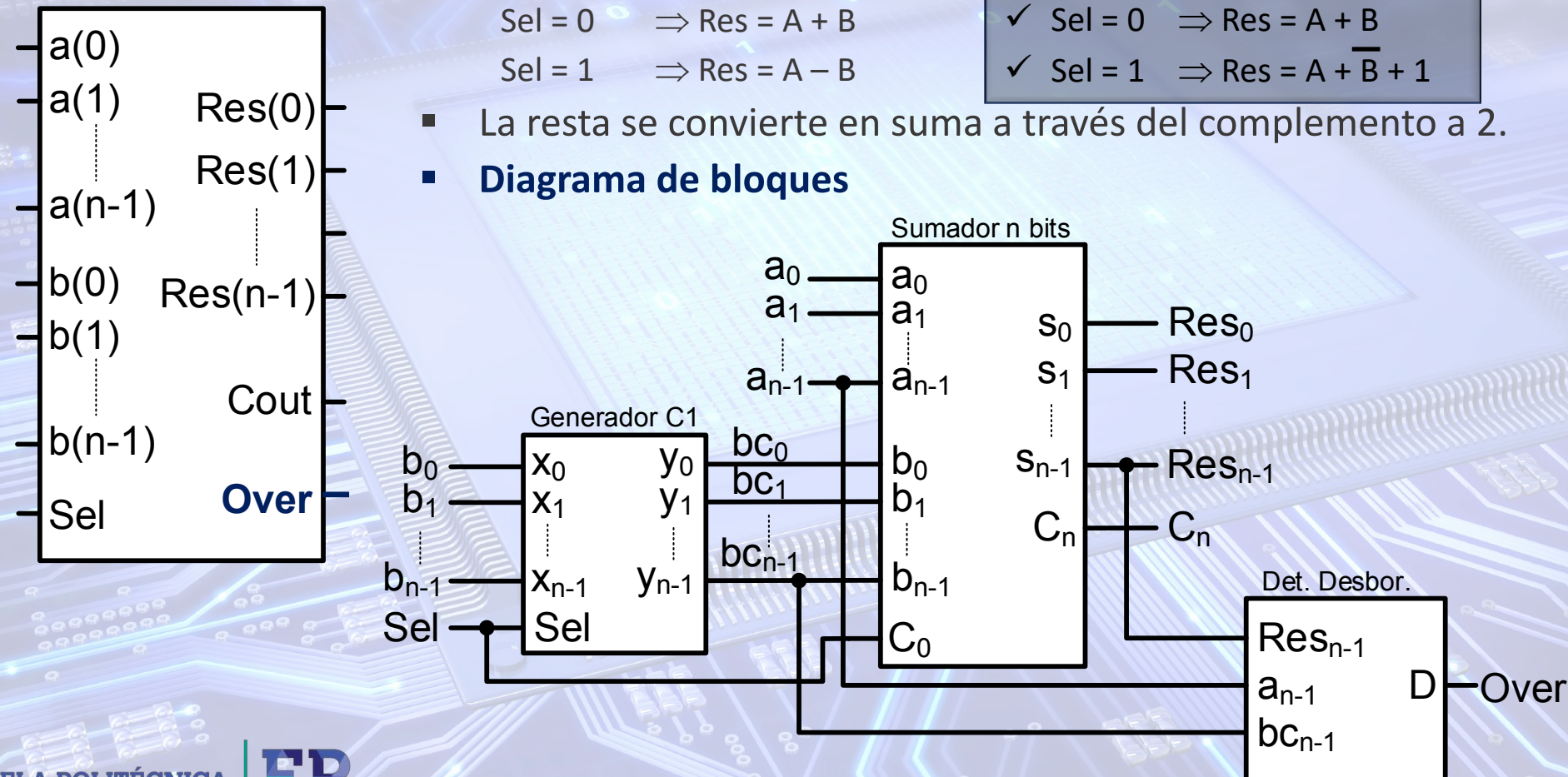
Sel = 1 $\Rightarrow$ Res = A - B

✓ Sel = 0 $\Rightarrow$ Res = A + B

✓ Sel = 1 $\Rightarrow$ Res = A + $\overline{B}$ + 1

■ La resta se convierte en suma a través del complemento a 2.

■ Diagrama de bloques



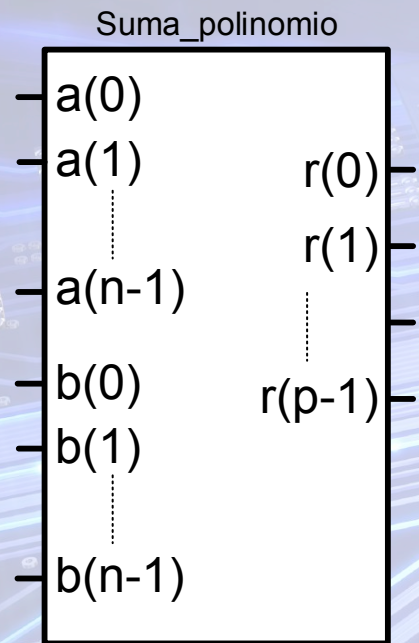
11. Problemas

8. Cálculo de un polinomio

- **A** y **B** son las entradas de n bits de los operandos de entrada.
- **R** es la salida de p bits en la que se representa el resultado de la operación aritmética **$A \cdot B + 3A + 5B$** , sin que se produzca desbordamiento, y teniendo en cuenta que $n \geq 3$.

- **Aclaraciones:**

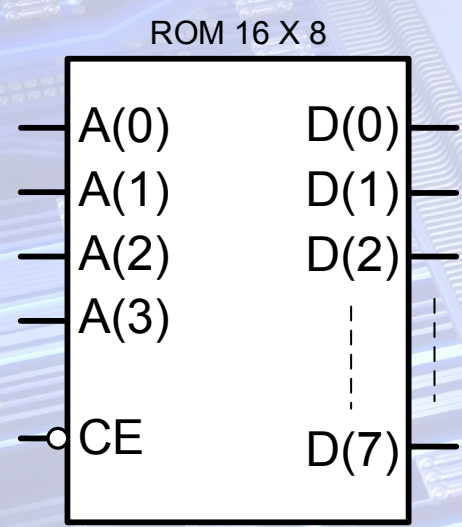
- ✓ Si A es de n_1 bits y B de n_2 bits, el producto de A por B , tiene n_1 más n_2 bits, es decir, $2 \cdot n$ bits.
- ✓ Considerando $A = B$, y sacando factor común a A :
 - $R = A \cdot (A+8)$
 - Dado que la suma de $A + 8$, puede generar un acarreo, el resultado de esa suma será de $n+1$ bits.
- ✓ Por tanto, el número de bits de la salida (p) será:
 - $P = n + (n + 1) = 2 \cdot n + 1$.



11. Problemas

9. Diseño de una ROM 16x8

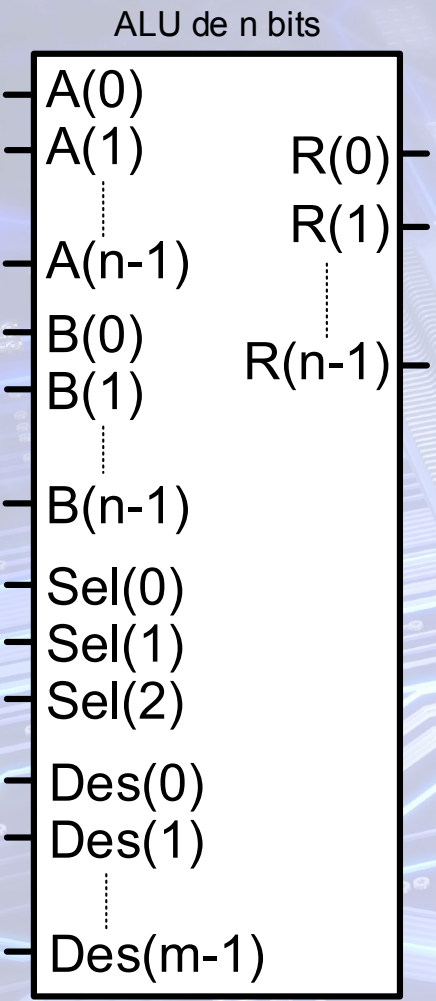
- Tiene 16 palabras de 8 bits cada una
 - ✓ Tendrá 4 patillas de direcciones (A) y 8 patillas de datos (D)
- La entrada CE es el Chip Enable o habilitación de chip
 - ✓ Es **activa a nivel lógico bajo** de forma que sólo se puede leer una palabra de la ROM si CE está activa.
 - ✓ Si está desactiva las patillas de datos estarán en alta impedancia.



Dirección (H)	Contenido	Dirección (H)	Contenido
0	0	8	87 ₁₆
1	3 ₁₆	9	44 ₁₆
2	45 ₁₆	A	B9 ₁₆
3	EF ₁₆	B	AA ₁₆
4	55 ₁₆	C	FF ₁₆
5	2C ₁₆	D	58 ₁₆
6	79 ₁₆	E	9D ₁₆
7	AD ₁₆	F	66 ₁₆

11. Problemas

10. Diseño 5. ALU de n bits



- **Por defecto** se considerará que es de **4 bits**
- La descripción debe ser **completamente estándar con el IEEE**
- Tiene que **realizar las operaciones** indicadas en la tabla, según el valor de las **entradas de selección de operación Sel**
- Para las **operaciones de desplazamiento** no se podrá usar el operador de concatenación.
 - ✓ Se usarán los operadores de desplazamiento lógicos correspondientes declarados en el **Package Numeric_std**
 - ✓ Se realizará el **número de desplazamientos** indicado por la entrada **Des**. Por defecto, se considerará, **m = 2**

Sel	Operación	Sel	Operación
000	AND lógica de A y B	100	A + 1
001	XOR lógica de A y B	101	A - 1
010	A + B	110	Desp. Lógico a la derecha de A
011	A - B	111	Desp. Lógico a la izquierda de A